IBM Innovation Centre for Education

NCS NihonCommunicationSolutions

Yenepoya Institute of Arts, Science, Commerce & Management.

Project- High Level Design

On

Identity Security Framework for Edtech

Student Name:

Khadeejath Mufeeda Kunhadukkam

23BSCED002

Table of Contents

1. Introduction ----------
   1.1. Scope of the Document ----------
   1.2. Intended Audience ----------
   1.3. System Overview ----------

2. System Design ----------
   2.1. Application Design ----------
   2.2. Process Flow ----------
   2.3. Information Flow ----------
   2.4. Components Design2.5. Key Design Considerations ----------
   2.6. API Catalogue ----------

3. Data Design ----------
   3.1. Data Model ----------
   3.2. Data Access Mechanism ----------
   3.3. Data Retention Policies ----------
   3.4. Data Migration ----------

4. Interfaces ----------

5. State and Session Management ----------

6. Caching ----------

7. Non-Functional Requirements ----------
   7.1. Security Aspects ----------
   7.2. Performance Aspects ----------

8. Reference ----------

INTRODUCTION

The Identity Security Framework for EdTech is designed to provide a secure and structured authentication and authorization system for educational platforms. With the increasing use of online learning systems, ensuring that only legitimate users access appropriate resources has become a critical requirement. This system focuses on protecting user identities, managing roles, and enforcing access control policies.
The framework ensures that users such as students, teachers, and administrators are properly verified and granted access based on their assigned roles. It also strengthens security by implementing authentication mechanisms, role-based access control (RBAC), and session management.

1.1 Purpose

The main purpose of this system is to design a secure identity management framework for an EdTech platform that authenticates users securely, controls access based on user roles, prevents unauthorized access to sensitive data, and maintains system integrity and user privacy.

1.2 Scope

The scope of this project includes secure login and registration system, role-based access control for Student, Teacher, and Admin, management of user identities and permissions, secure session handling after login, and basic monitoring and logging of user activity.
This system is applicable to any online educational platform requiring controlled access to learning resources and administrative features.

1.3 Objectives

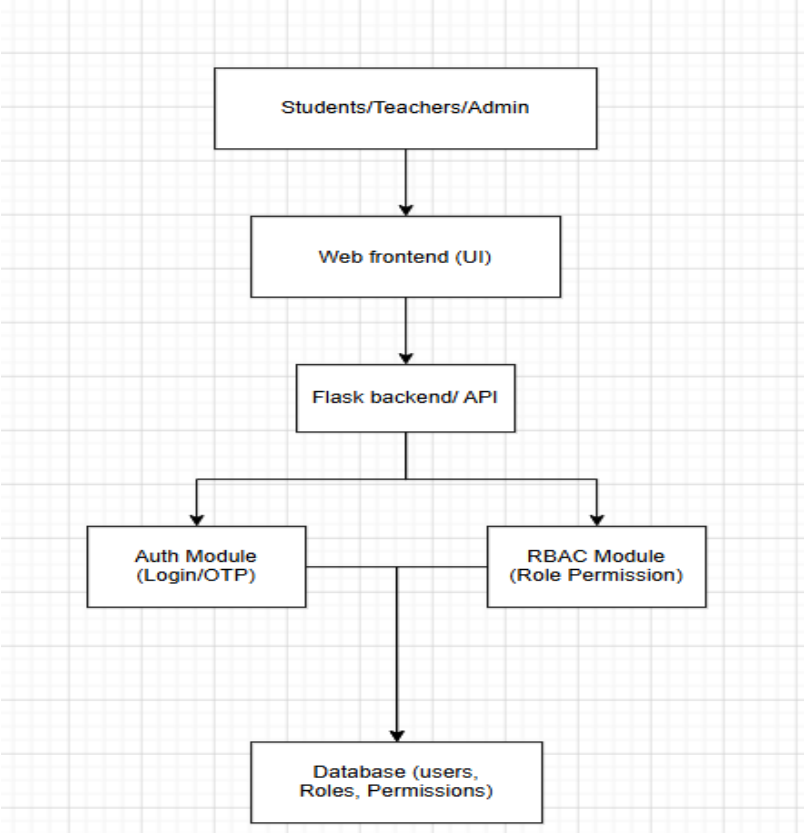
The key objectives of this system are to implement a secure authentication system for users, enforce Role-Based Access Control (RBAC), protect sensitive user and academic data, ensure only authorized access to system modules, and improve the overall security of EdTech platforms.

1.4 Motivation

The motivation behind this project is the increasing demand for secure online learning systems. Many educational platforms face issues such as unauthorized access, weak password protection, and lack of proper role management. This framework aims to solve these problems by introducing a structured identity security model.

1.5 System Overview

The system consists of a web-based EdTech platform where users interact through a frontend interface. The backend handles authentication, authorization, and database management. A security layer ensures that each user is validated and granted access according to their role.



2. System Design

2.1. Application Design

The system is designed as a web-based Identity Security Framework for an EdTech platform. It follows a client-server architecture where users interact through a web interface and all authentication, authorization, and role management are handled at the backend.

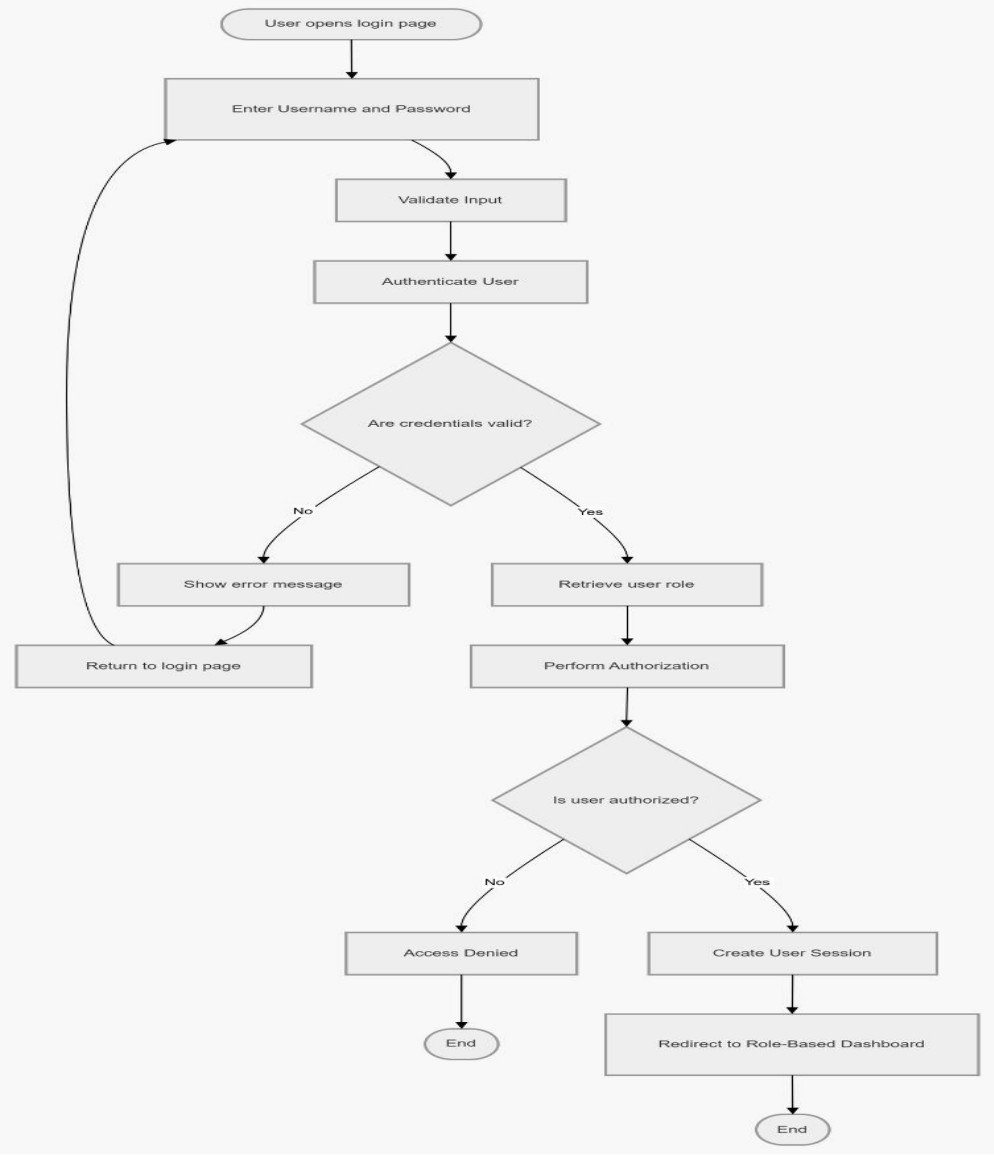
The application supports three main user roles: Student, Teacher, and Admin. Each user interacts with the system through a centralized login portal, and access is controlled using Role-Based Access Control (RBAC).

The backend is responsible for handling authentication (login/registration), validating user credentials, assigning roles, managing sessions, and enforcing access restrictions across different modules of the EdTech system.

2.2. Process Flow

The system follows a structured identity verification and access control flow:

• Step 1 — User Login Request: User enters username and password through the login interface
• Step 2 — Authentication: System verifies credentials using hashed password comparison
• Step 3 — User Validation: System checks whether the user exists in the database
• Step 4 — Role Retrieval: User role (Student / Teacher / Admin) is fetched from database
• Step 5 — Authorization (RBAC): System checks permissions based on assigned role
• Step 6 — Access Control Decision: System grants or denies access to requested resources
• Step 7 — Session Creation: Secure session is created for authenticated user
• Step 8 — Activity Logging: Login and actions are logged for security monitoring



2.3. Information Flow

Data flows through the system in the following direction:

• User Input (Login Credentials) → Web Frontend

• Frontend → Backend Authentication Module

• Authentication Module → User Database (Credential Verification)

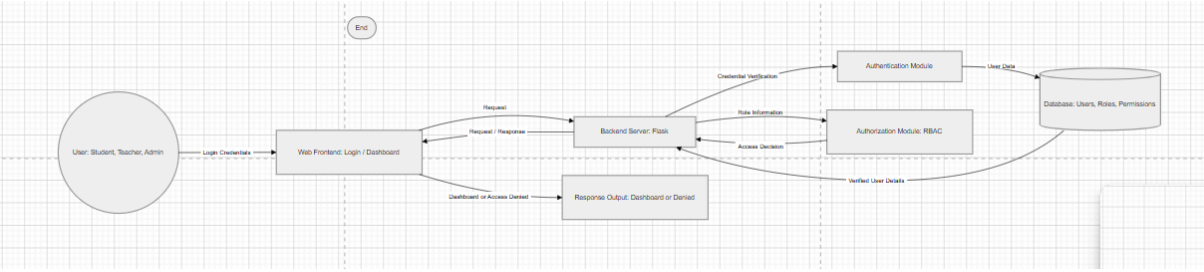
• Database → Role Information Returned

• Role → RBAC Authorization Module

• RBAC Module → Access Decision (Allow / Deny)

• System → Dashboard / Restricted Page

• All login events → Security Log



2.4. Components Design

The system is composed of the following major components

Component	Description
User Interface (Frontend)	Provides login, registration, and role-based dashboards for users
Authentication Module	Handles user login, password verification, and credential validation
Password Security Module	Stores and verifies hashed passwords using secure hashing algorithms
Authorization Module (RBAC)	Controls access based on user roles (Student, Teacher, Admin)
Session Management Module	Maintains user sessions after successful login
User Management Module	Handles creation, update, and deletion of user accounts

Database	Stores user credentials, roles, permissions, and session data
Logging & Monitoring Module	Tracks login attempts, access logs, and suspicious activities

2.5. Key Design Considerations

- The system implements Role-Based Access Control (RBAC) to ensure users only access permitted resources based on their assigned role (Student, Teacher, Admin).
- Passwords are securely stored using hashing techniques to prevent unauthorized access to user credentials.
- Session management is used to maintain secure user login sessions and prevent session hijacking.
- The principle of least privilege is enforced, ensuring users are given only the minimum required access.
- All authentication attempts and user activities are logged for monitoring and security auditing purposes. The system architecture is modular, allowing future scalability for additional roles or features.

2.6. API Catalogue

This system does not expose or consume external REST APIs. All data processing is internal. The following internal function interfaces are defined:

Function	Input	Output	Description
authenticate_user()	Username, Password	Success / Failure	Validates user credentials during login
hash_password()	hash_password()	Hashed Password	Converts password into secure hashed format
verify_password()	Password, Hash	Boolean	Compares entered password with stored hash
get_user_role()	User ID	Role (Student/Teacher/Admin)	Retrieves assigned user role from database
authorize_access()	Role, Resource	Allow / Deny	Controls access based on RBAC rules
create_session()	User ID	Session Token	Creates secure session after login

3. Data Design

3.1. Data Model

The system uses a relational data model to store and manage identity-related information for the EdTech platform. The primary data entities include users, roles, permissions, sessions, and activity logs.

Column	Data Type	Description
User_ id	Integer / UUID	Unique identifier for each user
username	String	Login name of the user
password_ hash	String	Securely stored hashed password
email	String	Registered email address
role_ id	Integer	Links user to a specific role
role_ name	String	Role type (Student, Teacher, Admin)
permission_ id	Integer	Defines access permissions
Session _id	String	Unique session identifier after login
Login _time	Timestamp	Time of successful login
Last_ activity	Timestamp	Tracks user activity
activity_ log	Text	Records user actions in system

3.2. Data Access Mechanism

• The system uses SQL-based queries (SQLite/MySQL) to retrieve and manage user identity data.

• All authentication requests are handled through backend functions such as authenticate_user() which validates credentials against stored hashed passwords.

• Role information is fetched from the database after successful authentication to determine user access level.

• Session data is stored temporarily during login sessions and validated on each request to ensure security.

• Access to sensitive data is restricted based on Role-Based Access Control (RBAC) rules.

3.3. Data Retention Policies

The system follows strict data retention and security policies:

• User Credentials: Stored permanently in the database in hashed format for authentication purposes.

• Session Data: Stored only for active sessions and automatically invalidated after logout or timeout.

• Activity Logs: Maintained for auditing and security monitoring purposes.

• Access Logs: Retained to track login attempts, both successful and failed.

• Data Privacy: Sensitive user information is protected and not exposed to unauthorized users or roles.

3.4. Data Migration

This system does not involve database migration. The dataset is a static flat file (KDDTrain+.txt). If the dataset is updated or replaced, the new file must be placed in the same project directory with the same filename, and the application must be restarted. Column indices used (4 and 41) must remain consistent with the NSL-KDD schema.

4. Interfaces

The system provides a web-based interface accessible through a browser (e.g., http://localhost:5000). The interface enables secure interaction between users and the identity security system, supporting authentication, role-based access, and user management.

UI Element	Type	Description
Login Page	HTML Form	Allows users to enter username and password for authentication
Registration Page	HTML Form	Enables new users to create an account securely
Dashboard	Dashboard	Displays role-based content for Student, Teacher, or Admin
User Management Page	Admin Interface	Allows admin to add, update, or delete users
Role Assignment Panel	Admin Interface	Enables assigning roles (Student, Teacher, Admin) to users
Course Access Page	Web Page	Provides controlled access to course content based on role
Navigation Bar	UI Component	Ends the session securely and redirects to login page
Logout Button	UI Component	Last 10 alert events with time, bytes, attack type, severity
Error Messages	UI Feedback	Displays authentication errors such as invalid credentials
Success Notifications	UI Feedback	Confirms successful login, registration, or actions

5. State and Session Management

The system uses session-based state management to maintain user authentication and control access across different parts of the application. Since the application follows a request-response model, session handling is essential to track logged-in users.

State is managed using server-side session mechanisms provided by the backend framework. The following components are used to maintain state during user interaction:

• session: Stores user-related information such as user_id and role after successful login
• user_id: Identifies the currently logged-in user
• user_role: Determines access permissions (Student, Teacher, Admin)
• login_status: Tracks whether the user is authenticated

The session is created after successful authentication and is validated on each request to ensure that only authorized users can access protected resources.

Key session management features include:

• Session Creation: A session is created when the user logs in successfully
• Session Validation: Each request checks session data to verify authentication
• Role-Based State Control: User role stored in session is used for RBAC enforcement
• Session Termination: Session is destroyed on logout to prevent unauthorized reuse
• Session Timeout: Sessions can expire after a period of inactivity for added security

All session data is handled securely to prevent session hijacking and unauthorized access. This ensures that user state is maintained consistently while preserving system security.

6. Caching

The following caching mechanisms are implemented:

• Frequently accessed non-sensitive data such as role definitions and permissions may be temporarily cached to reduce repeated database queries

• Static resources such as CSS, images, and frontend assets are cached at the browser level to improve page load speed

• Sensitive data such as passwords, session tokens, and authentication credentials are never cached

• Session-related data is managed using secure server-side sessions instead of caching mechanisms

• Database queries are optimized to minimize redundant access and improve response time

This controlled caching strategy ensures a balance between system performance and data security.

7. Non-Functional Requirements

7.1. Security Aspects

• User passwords are stored using secure hashing techniques to prevent unauthorized access

• Role-Based Access Control (RBAC) ensures that users can only access resources permitted to their role

• Session management mechanisms are implemented to prevent session hijacking and unauthorized reuse

• Input validation is applied to all user inputs to prevent injection attacks

• User activities such as login attempts and actions are logged for monitoring and auditing

• The system follows the principle of least privilege, granting only necessary permissions to users

• Secure logout functionality ensures proper session termination

7.2. Performance Aspects

The system provides fast authentication and authorization to ensure minimal delay for users

• Efficient database queries reduce response time during login and data retrieval

• Lightweight backend design minimizes server load and improves scalability

• Session handling is optimized to support multiple concurrent users

• Static resource caching improves frontend performance and reduces load times

• The system ensures smooth navigation between different modules without noticeable latency

8. References

• Flask Documentation — https://flask.palletsprojects.com/
• SQLite Documentation — https://www.sqlite.org/docs.html
• OWASP Security Guidelines — https://owasp.org/
• Role-Based Access Control (RBAC) Concepts
• Python Documentation — https://docs.python.org/3
• Web Application Security Best Practices